KisanRakshak Frontend - Work Summary

Prepared for: Team Leader

Date: 1 November 2026

Project: KisanRakshak, a Hindi-first web app for sugarcane and wheat farmers in Saharanpur district.

1. Purpose

This document summarises the frontend work completed so far, the order in which it was done, the current status, and what is still needed from the backend developer and the team.

2. Starting point

- The app screens already existed: home, farmer details (crop, growth stage, location, sowing date), dashboard (risks, AI explanation, weather, crop information), a speaker button, and Hindi/English switching.
- The app used demo data only. It was not connected to any backend.
- The agreed design: a Supabase Edge Function named crop-advisory at /functions/v1/crop-advisory. The frontend sends location, crop, sowing date (optional), growth stage and language. The backend returns weather, rule-based risks with severity, and an advisory.

3. Work log (in order)

Step 1 - API integration structure

- Added src/types/cropAdvisory.ts with request, success, failure and error-code types following the contract.
- Added src/services/advisoryContract.ts to build the request, check responses, and convert them to the dashboard's data shape.
- Added src/services/mockAdvisoryService.ts so demo data is kept separate from live data.
- Rewrote src/services/advisoryService.ts. fetchAdvisory() is the single entry point used by screens.
- Demo mode is the default. Setting VITE_USE_MOCK_ADVISORY to false selects live mode, which needs VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY.
- Updated the advisory hook (errors now carry a code), the shared types, the risk section (insufficient data and no-risk shown separately), the copy file and the dashboard page.
- Fixed two type errors. Typecheck and production build passed.

Step 2 - Zip export request

- A project zip was created to share the code. It could not be downloaded from the project, so the file was removed.
- Working method: click the project title (top left) and choose Download.

Step 3 - Final integration checklist (six tasks)

- Live API: real endpoint, base URL and anon key from environment variables, no silent fallback to demo data, 20-second timeout.
- HTTP mapping: 400 = invalid input, 404 = location not found, other failures = server error. Fixed a bug where a structured error body was read twice.
- Demo notice: shown only in demo mode. Weather source shown when the backend provides it.

- Rain probability: mapped the field precipitationProbabilityPct (camelCase, to match the contract style) to the existing rain field. Shows 'not available' only when the value is missing or null. One demo day has no value to test this.
- Fallback explanation: the explanation is shown whenever the backend provides it, even when the AI step failed. It is labelled 'Rule-based fallback' (Hindi and English). The frontend does not generate any advice.
- Optional sowing date: the field is optional in the input flow and left out of the request when empty. Crop info shows 'Not provided' when empty.
- Language: changing language sends a new request in the chosen language with a loading state. Crop, location, stage and date are kept.
- Refresh button is disabled while loading. A new request cancels the earlier one, so old results are ignored.
- Error states: separate Hindi and English messages for invalid input, location not found, server error, timeout, network failure, and malformed response. Each has a retry button.
- Typecheck and production build passed.

Step 4 - This summary

- Prepared as a PDF for the team leader.

4. Current status

- Frontend integration code is complete and type-checked, and the production build passes.
- Live backend: NOT tested. No working endpoint exists yet.
- Mobile layout: NOT visually checked. No browser is available in this environment.
- Demo data is still the default. The fallback label only appears in live mode.
- Hindi messages written during this work need review by a native speaker.

5. Files changed or added (this phase)

- Added: src/types/cropAdvisory.ts, src/services/mockAdvisoryService.ts.
- Rewritten or heavily updated: src/services/advisoryContract.ts, src/services/advisoryService.ts, src/pages/DashboardPage.tsx, src/i18n/copy.ts.
- Updated: src/hooks/useAdvisory.ts, src/types/farm.ts, src/components/dashboard/RiskSection.tsx, src/components/dashboard/WeatherPanel.tsx, src/components/dashboard/AiExplanationCard.tsx, src/components/dashboard/CropInfoCard.tsx, src/components/input/SowingDateStep.tsx, src/utils/farmerInputValidation.ts, src/data/mockWeather.ts.
- Removed: the temporary project zip file.

6. Decisions and assumptions

- Demo data is the default until the backend is ready.
- The browser never holds secret keys. Only the public anon key is used.
- Only backend-returned severity is shown. The frontend does not calculate risk.
- Backend text is shown in the language that was requested. The frontend does not translate it.
- Field name for rain probability chosen as precipitationProbabilityPct; the contract does not yet define it.

7. Items needing confirmation

- Rain probability field: add it to the contract and confirm the name.
- Fallback explanation: confirm that the advisory summary carries the rule-based text when AI fails.
- Sowing date: mark as optional in the contract.
- Weather source: the contract already defines weather.source (example value Open-Meteo). Confirm the backend always fills it.
- Growth-stage codes: the app sends IDs such as crown_root. The contract example uses codes such as CRI. The team must agree on one set.
- Sowing-date consistency warning: this was in the brief but does not exist in the app. Not built. Needs a decision on whether it is wanted.
- Error handling: confirm the error codes and messages agreed in the contract.
- Language re-request: the app re-requests on every language change. Confirm the backend supports this.
- Project name in README still shows a placeholder title; awaiting approval to change it to KisanRakshak.
- Who validates the crop risk thresholds (only the IMD wheat rule is marked as validated).

8. Verification results

- npm run typecheck: passed.
- npm run build: passed.
- Live endpoint test: not performed.
- Mobile layout check: not performed.